FitPot Build Guide

A step-by-step record of how FitPot is built and deployed, from an empty GitHub repo to the finished app. Every phase lists **what was built**, **what you do** (clicks and pastes), **how to check it worked**, and **fixes** for problems we hit.

- **Repo:** https://github.com/KritiCParikh/fitpot-grind
- **Live app:** https://kriticparikh.github.io/fitpot-grind/
- **Firebase project:** `fitpot-grind` (Spark / free plan)
- **What everything is and how it connects:** HOW_IT_WORKS.md

Phase	What	Status
0	Repo, PWA shell, Google sign-in, auto-deploy	✅ Done
1	Groups + passcode join	✅ Done
2	Daily check-in + calendar + streaks	✅ Done
3	Pot ledger, balances, board, rest days, penalty changes	✅ Done
4	Camera-only photos → Google Drive, photo wall	✅ Done
4.5	Group rules: weekly workout target, photo-required check-in, undo	✅ Done
5	Me tab: food logging (INDB + USDA + Open Food Facts), barcode, recipes, targets, weight, progress photos	✅ Done
5.1	Each member's own timezone; pot currency (₹ $ £ C$) with conversion	✅ Done
5.2	Me-first sign-in, multiple groups (one check-in for all, Leave), Training tab (progressive overload), measurements, export	✅ Built, ready to deploy
5.3	Email + password sign-in, verification, Account page (name, password, delete account)	✅ Built, ready to deploy
6	Vision model (scratch → fine-tune → browser)	⏳ Next
7	Language model nudges + push notifications	
8	Personalised training plans from your own model (uses the Training data)	

Stack at a glance

Layer	Tool	What it does	Cost
Frontend	React + Vite, as a PWA	The screens you tap; installable on phones	Free
Hosting	GitHub Pages	Serves the website	Free
Version control + deploy	GitHub + GitHub Actions	Stores code; every push to `main` rebuilds and redeploys	Free
Login	Firebase Authentication (Google)	"Sign in with Google"	Free
Database	Cloud Firestore	Groups, members, check-ins	Free tier
Backend	Google Apps Script	Saves photos to Drive (phase 4), scheduled jobs	Free
File storage	Google Drive	Workout photos in `FitPot/<member>/<month>/`	Free (15 GB)
ML training	Google Colab	Training the vision and language models	Free GPUs

Will Firebase ever charge? Not on the Spark plan. It has no billing account or card, so it can't bill you. If a free limit were ever exceeded, that part of the app would pause until the daily quota resets. Charges are only possible if you upgrade to the Blaze plan yourself.

Phase 0 — Repo, app shell, sign-in, auto-deploy

What was built

- React + Vite app configured as a PWA (installable, own icon, full screen).
- Google sign-in screen, Today screen, bottom tabs.
- `.github/workflows/deploy.yml`: builds the app and publishes it to GitHub Pages on every push to `main`.
- `firestore.rules`, `apps-script/Code.gs` (Drive backend starter), `ml/README.md`.

Step 1 — Create the GitHub repo and upload the code

1. On github.com, create a new repo (`fitpot-grind`). No README, no .gitignore.
2. Upload the project files. Either:

- **Command line:**

```bash
cd fitpot
git remote add origin https://github.com/KritiCParikh/fitpot-grind.git
git push -u origin main
```

- **Website:** repo → **Add file → Upload files** → drag the files in → **Commit changes.**

3. ⚠️ Drag-and-drop skips folders starting with a dot (`.github`). If the **Actions** tab later says "Get started with GitHub Actions", see Step 6.

Step 2 — Turn on GitHub Pages

Repo → **Settings → Pages → Source → GitHub Actions.**

Step 3 — Create the Firebase project

At console.firebase.google.com:

1. **Add project** → name it (`fitpot-grind`) → free Spark plan. Analytics optional.
2. **Authentication → Get started → Google → Enable** → pick a support email → **Save.**
3. **Authentication → Settings → Authorized domains → Add domain** → `kriticparikh.github.io`.
4. **Firestore Database → Create database** → production mode → a US region → **Create.**
5. **Firestore Database → Rules** tab → select all (Ctrl/Cmd + A) → delete → paste the full contents of `firestore.rules` → **Publish.**
    - Firebase's default rules (`allow read, write: if false;`) block everything. Replacing them is required.

Step 4 — Get the Firebase web config

1. Left sidebar → **Settings** (under Project Overview). Direct link: `https://console.firebase.google.com/project/fitpot-grind/settings/general`
2. Scroll to **Your apps** → click `</>` (Web).
    - No `</>` icon? Go to **Project Overview** → **+ Add app** → **Web.**
3. Nickname `fitpot-web` → leave "Firebase Hosting" unchecked → **Register app.**
4. You'll see:

```js
```

```
const firebaseConfig = {
  apiKey: "AIza...",
  authDomain: "fitpot-grind.firebaseapp.com",
  projectId: "fitpot-grind",
  storageBucket: "fitpot-grind.firebasestorage.app",
  messagingSenderId: "1234567890",
  appId: "1:1234567890:web:abc123"
};
```

You can always find it again under **Settings → Your apps**.

Step 5 — Add the config as GitHub secrets

1. Repo → **Settings** (repo tab, not your profile) → left sidebar **Secrets and variables → Actions**.
2. **New repository secret** → fill **Name** and **Secret** → **Add secret**. Repeat 6 times.
3. **Secret** = only what's inside the quotes. No quotes, no comma, no spaces.

Name	Secret (from your config)
`VITE_FIREBASE_API_KEY`	apiKey
`VITE_FIREBASE_AUTH_DOMAIN`	authDomain
`VITE_FIREBASE_PROJECT_ID`	projectId
`VITE_FIREBASE_STORAGE_BUCKET`	storageBucket
`VITE_FIREBASE_MESSAGING_SENDER_ID`	messagingSenderId
`VITE_FIREBASE_APP_ID`	appId

Example (made-up values):

```
Name:    VITE_FIREBASE_API_KEY
Secret:  AIzaSyD4x9Kq2mPl7wR8tYvB3nC6hJ1fG0eZkQs

Name:    VITE_FIREBASE_AUTH_DOMAIN
Secret:  fitpot-grind.firebaseapp.com
```

These values are safe inside a web app; the Firestore rules are what protect the data. Secrets just keep them out of the code.

Step 6 — Make sure the deploy workflow exists

Open the **Actions** tab.

- If you see **"Get started with GitHub Actions"**, the `.github` folder didn't upload. Click **set up a workflow yourself** → name the file `deploy.yml` → replace the editor contents with the workflow at the end of this guide (Appendix A) → **Commit changes**.

Step 7 — Deploy

Actions → Deploy to GitHub Pages → Run workflow. About a minute later it turns green.

Step 8 — Open and install

- Open **https://kriticparikh.github.io/fitpot-grind/** in the phone's **browser** (the GitHub app only shows code).
- **iPhone (Safari):** Share → **Add to Home Screen**.
- **Android (Chrome):** ⋮ → **Install app**.
- It opens full screen from its own icon.

Check it worked

- Login screen with the lime **FitPot** logo → **Sign in with Google** → Today screen.

Phase 0 fixes

Problem	Fix
Actions shows "Get started with GitHub Actions"	`.github` folder missing. Step 6.
Build fails with **exit code 254**	npm can't find `package.json` at the repo root — files were uploaded inside a `fitpot/` folder. In `deploy.yml`, under the build job's `runs-on: ubuntu-latest`, add `defaults: run: working-directory: fitpot`, and change `path: dist` to `path: fitpot/dist`. (Appendix A shows both.)
"Node.js 20 is deprecated" / "ubuntu-latest will migrate"	Harmless GitHub notices. Ignore.
"Firebase isn't configured" screen	Secrets missing or misnamed. Check all 6 names exactly (Step 5), then re-run the workflow (Step 7).
Sign-in error `auth/unauthorized-domain`	Step 3.3: add `kriticparikh.github.io` to Authorized domains.
Already have a site at `kriticparikh.github.io`	No conflict. FitPot lives at `/fitpot-grind/`; the main site is untouched.

Phase 1 — Groups + passcode join

What was built

- **Onboarding screen** after sign-in (since Phase 5.2 this lives in the **Group** tab, and sign-in lands on **Me**): **Join group** (enter a 6-character passcode) or **Create group** (name + daily penalty).
- Creating a group generates a passcode like `K7QM3X` (no look-alike characters such as 0/O or 1/I).
- **Group tab:** big passcode, **Invite friends** button (share sheet on phones, copies a message on desktop), penalty setting, member list with join dates, **Switch group**.
- Group timezone is saved at creation (from the creator's phone). Since Phase 5.1 it's only a fallback: each member's day follows their own phone's timezone.

How the data is stored (Firestore)

```
users/{uid}                          name, photoURL, groupId (group you're
viewing), groupIds (all your groups, phase 5.2), leftGroupIds (5.3)
passcodes/{code}                     groupId
groups/{groupId}                     name, passcode, penalty, timezone, createdBy,
createdAt
groups/{groupId}/members/{uid}       name, photoURL, joinedAt (server time)
groups/{groupId}/checkins/{date_uid}   uid, date, createdAt (server time)
groups/{groupId}/restdays/{date}       votes {uid: server time}        (phase 3)
groups/{groupId}/penalties/{id}        penalty, effectiveDate, setBy, setAt
(phase 3)
groups/{groupId}/photos/{date_uid}     uid, date, fileId (Google Drive),
createdAt (phase 4)
groups/{groupId}/targets/{id}          target (days/week), effectiveDate (a
Monday), setBy, setAt (phase 4.5)
groups/{groupId}/tzchanges/{id}        uid, timezone, effectiveDate, setAt
(phase 5.1)
groups/{groupId}/currencies/{id}       from, currency, rate, penalty,
effectiveDate, setBy, setAt (phase 5.1)
```

Groups also store `weeklyTarget` (1–7) and `photoRequired` (true/false) since phase 4.5, and `currency` since phase 5.1. Members store their own `timezone` since phase 5.1.

Security built into the rules

- Only group members can read the group, its members and check-ins.
- You can only join with the real passcode, and only add yourself.
- Passcodes can be looked up one at a time but never listed.
- Join time is stamped by Google's server and can't be changed later, so nobody can fake a later join date to dodge penalties.

What you do

Step 9 — Update the Firestore rules. Firebase → **Firestore Database → Rules** → select all → delete → paste the new `firestore.rules` → **Publish**. The rules changed in this phase; the app won't work with the old ones.

Step 10 — Upload the new code. See "Uploading an update" below.

Check it worked

1. Sign in → you see **Join group** / **Create group**.
2. Create a group ("Morning Grinders", $5) → you land on Today.

3. **Group** tab → passcode shows → **Invite friends**.
4. On a second Google account (or a friend's phone): sign in → **Join group** → enter the passcode → they land in the same group and appear in the member list.

Phase 2 — Daily check-in + calendar

What was built

- **Today tab**
  - Big **CHECK IN** button with a pop animation and a short vibration on Android. Turns into a ✓ once done.
  - **At stake now:** penalty × members who haven't checked in yet.
  - **If it ended now:** what each person who showed up would earn.
  - **Crew today:** everyone's live status (In ✓ / Not yet), updating in real time as friends check in.
- **Calendar tab**
  - Month grid: **green** = you checked in, **red** = missed, outlined = today (still open), grey = before you joined or future.
  - Tap any day → who checked in, who missed, that day's pot and per-person split.
  - **Current streak** 🔥 (not broken until midnight), **best streak**, **total** workouts.
  - Browse back through earlier months.

Anti-cheat

- One check-in per person per day (the database refuses a second one).
- Check-ins can't be edited or deleted.
- Each check-in is stamped with **Google's server time**, not the phone's clock. A check-in only counts if that server time falls on the right day in the group's timezone, so changing your phone's clock can't backdate one.

What you do

Same as Phase 1 (Steps 9–10). Phases 1 and 2 ship together.

Check it worked

1. Today → tap **CHECK IN** → button becomes ✓, you show as **In** ✓.
2. On a friend's phone, their Today screen updates without refreshing.
3. Calendar → today is green, streak shows 1🔥.
4. Tap today → the day breakdown shows the pot and split.

Phase 1–2 fixes

Problem	Fix
"Missing or insufficient permissions"	The Firestore rules weren't updated. Step 9.
"No group with that passcode"	Check the code (letters are uppercase, no 0/O/1/I).
Stuck on "Couldn't open your group"	Tap **Choose a group** and re-join with the passcode.
Sign-in popup doesn't open in the installed iPhone app	Open the site once in Safari and sign in there, then use the home-screen app.

Phase 3 — Pot ledger, balances, board, rest days

What was built

- **Automatic ledger.** Every finished day is settled from the check-ins; nobody types in amounts and no balance can be edited. The rules for each day:
  - Everyone who missed owes that day's penalty.
  - The pot (missed × penalty) is split evenly among everyone who checked in.
  - If **nobody** checked in, nobody is charged (there's no one to pay).
  - People aren't charged for days before they joined.
  - Amounts are rounded to the cent, so a settle-up can be off by a cent.
- **Board tab** (replaces "coming soon")
  - **Balances:** everyone's net for the month, highest first. The last place gets a 🙈 (the shame board).
  - **Settle up:** who pays whom, using the fewest possible transfers.
  - **Streaks:** current streak and best streak for everyone (looks back 60 days).
  - Totals: pot moved, days settled, rest days. Browse back to earlier months.
  - This month is settled **through yesterday**; today joins the ledger after midnight.
- **Today tab additions: Your month** balance, and a **Rest day?** card.
- **Rest-day voting.** Tap **Vote rest**. If more than half the group votes before midnight, the day becomes a rest day and nobody pays. Votes are stamped with Google's server time; a vote cast after the day ends doesn't count. You can take your vote back.
- **Penalty changes** (Group tab). A new penalty always starts **tomorrow**, so today's stakes never change mid-day and past days keep the penalty they had. Every change is logged permanently.

- **Calendar:** rest days show in grey; tapping a day shows that day's penalty and each person's +/− amount.

Worked example (from the blueprint)

5 friends, $20 penalty. 3 check in, 2 don't → pot = 2 × $20 = **$40** → each of the 3 gets **+$13.33**, each of the 2 gets **−$20**.

What you do

Step 11 — Update the Firestore rules. Firebase → **Firestore Database → Rules** → select all → delete → paste the new `firestore.rules` → **Publish**. New in this phase: rest-day votes and penalty changes. The group's base penalty can no longer be edited directly.

Step 12 — Upload the new code. Same as before ("Uploading an update" below): drag in `src`, `docs`, `firestore.rules`, `README.md` → commit.

Check it worked

1. **Board** tab opens and shows balances (all $0.00 on your first day; real numbers appear after midnight).
2. **Today** → **Vote rest** → the button shows **Voted** ✓ and the count goes up. Tap again to take it back.
3. **Group** → set a new penalty → it shows "→ $X from tomorrow"; today's "At stake" doesn't change.
4. The next day, **Board** shows yesterday's result, and **Calendar** → tap yesterday shows the +/− amounts.

Phase 3 fixes

Problem	Fix
"Missing or insufficient permissions" when voting or changing the penalty	Rules not updated. Step 11.
Balances are all $0.00	Normal on day one. Balances cover finished days only (through yesterday).
Settle-up is off by $0.01	Cent rounding when a pot doesn't split evenly.

Phase 4 — Camera-only photos → Google Drive, photo wall

What was built

- 📸 **Add workout photo** button on Today (highlighted once you've checked in), and a **Wall** tab.
- **Camera-only.** The app opens the live camera. There's no file picker anywhere, so gallery or old photos can't be posted. Works on phones and on laptops with a webcam. **Flip** switches front/back camera; **Retake** before posting.
- **Watermark** burned into every photo: `FitPot · <name> · <date, time>`.
- **Compressed** to about 300–500 KB (max 1080 px) before upload, so Drive space lasts years.
- **One-time consent screen** before the first photo: shared with the whole group, saved in the organiser's Drive, time-stamped, no location.
- **Saved to Google Drive** of whoever deploys the script: `FitPot/<member name>/<YYYY-MM>/<YYYY-MM-DD_HHmmss>.jpg`.
- **Photo wall:** today's photos by default, ‹ › to browse earlier days. Each card shows the member and time.
- Photos are **private in Drive**. The app fetches them through the script, which only hands them to members of the same group, and caches them on the phone so each downloads once.
- One photo per person per day, today only, can't be edited or backdated (server time again).
- Photos are **optional**. Check-in alone still counts.

How it works

```
Phone camera → watermark + compress → Apps Script (your Google account)
   ↳ checks the Firebase login is real
   ↳ checks, through Firestore's own rules, that the person is in the group
   ↳ saves the JPEG into Drive: FitPot/<name>/<month>/
   ↳ returns the Drive file id
App → writes groups/{id}/photos/{date_uid} pointing at that file id
Wall → asks the script for each photo → shown to group members only
```

What you do

Step 13 — Update the Firestore rules. Firebase → **Firestore Database → Rules** → select all → delete → paste the new `firestore.rules` → **Publish.** (Adds the `photos` collection.)

Step 14 — Create the Apps Script. Use the Google account whose Drive should hold the photos.

1. Go to **script.google.com** → **New project**.
2. Click the title "Untitled project" at the top → rename it `FitPot Drive`.
3. In the editor, `Code.gs` is open. Select all → delete → paste the full contents of `apps-script/Code.gs` from the repo → **Save** (💾 or Ctrl/Cmd + S).

Step 15 — Add the script properties.

1. Left sidebar → ⚙️ **Project Settings**.
2. Scroll to **Script Properties** → **Add script property**. Add three:

Property	Value
`FIREBASE_API_KEY`	same as your `VITE_FIREBASE_API_KEY` (the `apiKey` from Firebase)
`FIREBASE_PROJECT_ID`	`fitpot-grind`
`TIMEZONE`	`America/New_York`

3. **Save script properties**.

Step 16 — Deploy it as a web app.

1. Top right → **Deploy** → **New deployment**.
2. Click the ⚙️ next to "Select type" → **Web app**.
3. Description: `FitPot v1`. **Execute as:** `Me`. **Who has access:** `Anyone`.
4. **Deploy** → **Authorize access** → pick your Google account.
5. You'll see "Google hasn't verified this app". This is normal for your own script. Click **Advanced** → **Go to FitPot Drive (unsafe)** → **Allow**. (It's asking permission for *your* script to use *your* Drive.)
6. Copy the **Web app URL** (ends in `/exec`).
7. Test it: paste the URL into a browser tab. You should see `{"ok":true,"service":"fitpot-drive","configured":true}`. If `configured` is `false`, recheck Step 15.

Step 17 — Give the URL to the app.

1. GitHub repo → **Settings** → **Secrets and variables** → **Actions** → **New repository secret**.
2. **Name:** `VITE_DRIVE_UPLOAD_URL` **Secret:** the `/exec` URL → **Add secret**.

Step 18 — Upload the new code (see "Uploading an update"): drag in `src`, `docs`, `apps-script`, `firestore.rules`, `README.md` → commit. The deploy runs automatically and picks up the new secret.

Check it worked

1. **Wall** tab no longer says "Photos aren't switched on yet".
2. **Today** → 📸 **Add workout photo** → consent screen → **I understand** → allow camera → take a photo → **Post to group**.
3. The photo shows on the **Wall**, with the watermark at the bottom.
4. In Google Drive: **FitPot** → **<your name>** → **<this month>** has the JPEG.
5. A friend in the group sees your photo on their Wall.

Changing the script later

Edit `Code.gs` → **Save** → **Deploy** → **Manage deployments** → ✏️ edit the existing deployment → **Version: New version** → **Deploy**. The URL stays the same, so nothing changes on GitHub. (A *new* deployment would give a new URL.)

Phase 4 fixes

Problem	Fix
Wall says "Photos aren't switched on yet"	`VITE_DRIVE_UPLOAD_URL` secret missing, or the deploy ran before you added it. Step 17, then **Actions** → **Run workflow**.
"Camera permission was denied"	iPhone: **Settings** → **Safari** → **Camera** → **Allow**. Android Chrome: tap the 🔒 by the address → **Permissions** → **Camera** → **Allow**. Then reopen the app.
"Not signed in (token rejected)"	`FIREBASE_API_KEY` script property is wrong. Step 15.
"Not a member of this group"	`FIREBASE_PROJECT_ID` is wrong, or the Firestore rules weren't updated (Step 13).
Upload fails / "Failed to fetch"	Deployment access isn't **Anyone**, or the URL doesn't end in `/exec`. Redo Step 16.
Photo card says "Couldn't load"	Script was redeployed as a new deployment (new URL). Update the secret or edit the existing deployment instead.
Browser URL test shows a Google sign-in page	**Who has access** isn't set to **Anyone**.

Phase 4.5 — Group rules: weekly target, photo-required check-in, undo

What changed

1. Workout days per week (group rule). Set in **Group → Group rules**, or when creating a group.

- **Every day (7)** works as before: each day settles at midnight.
- **1–6 days a week** switches to weekly settling:
  - Weeks run **Monday → Sunday** and settle at **Sunday midnight**.
  - Anyone under the target pays **penalty × days short**.
  - The pot is split evenly among everyone who **hit** the target. If nobody hit it, nobody pays.
  - Each group **rest day** that week lowers the target by 1.
  - Someone who joins mid-week starts counting next Monday (their first partial week is free). Same for a brand-new group.
  - A week's money counts in the month the week **ends**.
- Changing the target **starts next Monday**; the current week never changes. Changes are logged permanently.

Worked example. Target 4, $5. Over one week: Asha 5 workouts, Ben 2, Cy 4. Ben is 2 short → pays 2 × $5 = **$10**. Asha and Cy hit 4 → **+$5 each**.

2. Photo-required check-in (group rule, on by default).

- **CHECK IN** opens the camera; posting the photo *is* the check-in (one step: **Check in + post**).
- The database enforces it: a check-in without that day's photo is rejected.
- Turn it off in **Group → Group rules** to make photos optional again.
- If it's on but the Drive script isn't set up yet (Phase 4), the CHECK IN button is disabled with a note. Finish Phase 4 or turn the rule off.

3. Undo check-in. After checking in, **Undo check-in** (small link under the button) removes today's check-in and its wall photo. Only for today; you can check in again afterwards. The photo file stays in Drive.

Screens

- **Today** (weekly groups): **Your week** (e.g. 2/4), **Days left**, **Your month**; the crew list shows each person's weekly count. A line under the rest-day card states the group's rule.
- **Board**: a **Weeks** list with each settled week's target, pot, and everyone's count (✓ hit / ✗ short).

- **Calendar**: in weekly groups, tapping a day shows who worked out; money shows on the Board.

What you do

Step 19 — Update the Firestore rules. Firebase → **Firestore Database → Rules** → select all → delete → paste the new `firestore.rules` → **Publish.** (Adds photo-required enforcement, undo, and the `targets` log.)

Step 20 — Upload the new code. Drag in `src`, `docs`, `firestore.rules`, `README.md` → commit.

Step 21 — Set your group's rule. In the app: **Group → Group rules → Change target → 4 days a week.** It starts next Monday. Groups created before this update default to **every day** (daily) and **photo required**.

Check it worked

1. **Group** tab shows **Group rules** with the target, the photo toggle, and the penalty.
2. After setting 4 days: it shows "every day → 4 days a week from Monday"; from Monday, Today shows **Your week 0/4**.
3. **CHECK IN** opens the camera (photo rule on) → **Check in + post** → ✓ and the photo is on the Wall.
4. **Undo check-in** → confirm → the button returns to CHECK IN and the photo leaves the Wall.

Phase 4.5 fixes

Problem	Fix
CHECK IN is greyed out with a note about photos	Photo rule is on but the Drive script isn't deployed. Do Phase 4, or turn the photo rule off in Group.
"Missing or insufficient permissions" on check-in, undo, or changing the target	Rules not updated. Step 19.
Target change doesn't show on Today	Normal. It starts next Monday.
Undo is gone the next day	By design. Only today's check-in can be undone.

Phase 5 — Me tab: food, targets, weight, progress photos

Everything in **Me** is **optional** and **private**: only you can see it (Firestore `private/{you}/…`, owner-only). It never affects check-ins or the pot.

What was built

- **New "Me" tab** (bottom bar: Today · Calendar · Wall · Board · **Me** · Group) with three sections:

🍽️ Food

- Diary by **Breakfast** / **Lunch** / **Dinner** / **Snacks**, with ‹ › to see other days.
- **Daily totals** (calories, protein, carbs, fat, fibre) with progress bars against your targets (calories turn orange when you go over).
- **+ Add** opens the food finder:
  - **Recent**: foods you've logged, one tap to re-log with your last portion.
  - **My foods & recipes**: your saved foods.
  - **Search** across three databases, toggled with chips:
    - 🇮🇳 **India**: Indian Nutrient Databank (INDB), 1,014 common Indian dishes, bundled in the app (instant, works offline).
    - 🇺🇸 **USA**: USDA FoodData Central (generic foods, US brands, mixed dishes).
    - 📦 **Packaged**: Open Food Facts (branded products worldwide, incl. Indian brands).
  - ▦ **Scan barcode**: packaged foods. If the product isn't found, you enter the label once and it's saved to My foods, so it scans instantly next time.
  - 📷 **Scan plate · soon**: placeholder for your own vision model (phase 6, no AI APIs).
- **Portion**: amount × unit (g, 100 g, bowl/plate/piece/serving from the database), live nutrition preview, pick the meal, **Add**.
- ⭐ **Save to My foods** from any search result.
- **My foods & recipes**: create your own food (per serving, e.g. from a label), or build a **recipe** from ingredients (e.g. "Chicken curry (home)", makes 4 servings), which is then logged like any food.

⚖️ Progress

- **Manual weight log**: weight + date (past dates allowed; logging a date again replaces it). kg or lb.
- **Trend chart** (30 d / 90 d / 1 y / all), tap or hover for exact values, dashed line for your goal weight.
- **Latest**, **7-day** and **30-day** change; full entry list with delete.
- 📸 **Progress photos (optional)**: saved to **your own Google Drive** in a `FitPot Progress` folder, never to the organiser's Drive, never visible to the group. Camera **or** gallery. Tap

two photos to compare side by side. FitPot can only access files it created there, nothing else in your Drive.

🎯 Targets

- Your own daily calories, protein, carbs, fat, fibre (all optional), weight unit, goal weight. AI-suggested targets come later from your own model.

What you do

Step 22 — (Recommended) Get a free USDA key. Without it, USA search uses a shared demo key that runs out after a few searches an hour.

1. Go to **api.data.gov/signup** → fill in name + email → **Sign up**.
2. The key arrives on screen and by email (a long string).
3. GitHub repo → **Settings → Secrets and variables → Actions → New repository secret** → **Name:** `VITE_USDA_API_KEY` **Secret:** the key → **Add secret**.

Step 23 — (For progress photos) Create a Google sign-in client. Your Firebase project is also a Google Cloud project; we add Drive access to it.

1. Go to **console.cloud.google.com**. Top bar → project picker → choose **fitpot-grind**.
2. **Turn on the Drive API:** ≡ menu → **APIs & Services → Library** → search **Google Drive API** → **Enable**.
3. **Consent screen:** ≡ → **APIs & Services → OAuth consent screen** (may be called **Google Auth Platform**).
   - If it says **Get started**: App name `FitPot`, your support email → **Audience: External** → contact email → agree → **Create**. (If Firebase already created one, just continue.)
4. **Scope: Data Access → Add or remove scopes** → find `…/auth/drive.file` ("See, edit, create, and delete only the specific Google Drive files you use with this app") → tick → **Update → Save**.
5. **Publish: Audience → Publish app** → confirm. `drive.file` is a non-sensitive scope, so no Google review is needed. (If you leave it in *Testing*, only people you list as test users can connect, and their access expires every 7 days.)
6. **Client: Clients** (or **Credentials → Create credentials → OAuth client ID**) → **Application type: Web application** → Name `FitPot web` → **Authorized JavaScript origins → Add URI** → `https://kriticparikh.github.io` (and `http://localhost:5173` if you run it locally) → **Create**.
7. Copy the **Client ID** (ends in `.apps.googleusercontent.com`). It's not a secret, but we store it like the others.

8. GitHub → **New repository secret** → **Name:** `VITE_GOOGLE_CLIENT_ID` **Secret:** the Client ID → **Add secret**.

Step 24 — Let the build use the two new secrets. Your `deploy.yml` was created by hand on GitHub, so add two lines to it:

1. GitHub repo → open `.github/workflows/deploy.yml` → ✏️ **Edit**.
2. Find the line `VITE_DRIVE_UPLOAD_URL: ${{ secrets.VITE_DRIVE_UPLOAD_URL }}`. Directly under it, with **exactly the same indentation**, add:

```yaml
          VITE_USDA_API_KEY: ${{ secrets.VITE_USDA_API_KEY }}
          VITE_GOOGLE_CLIENT_ID: ${{ secrets.VITE_GOOGLE_CLIENT_ID }}
```

3. **Commit changes**. (Appendix A shows the full file.)

Step 25 — Upload the new code. This update adds a library (barcode scanner) and a data file, so it includes more than usual. Drag in: `src`, `public`, `docs`, `package.json`, `package-lock.json`, `vite.config.js`, `README.md` → **Commit changes. No Firestore rules change** this time (the private area was already owner-only).

Check it worked

1. Bottom bar shows **Me**.
2. **Me → Targets** → set e.g. 2200 kcal, 120 g protein → **Save**.
3. **Me → Food → Breakfast + Add** → type `poha` → 🇮🇳 results appear → pick one → 1 bowl → **Add to Breakfast** → totals update.
4. Type `oats` → 🇺🇸 and 📦 results appear too.
5. ▦ **Scan barcode** on any pack → it's found, or you're offered **Create food** with the barcode filled in.
6. **Me → Progress** → save a weight → it appears; save another date → the chart draws.
7. **Connect my Google Drive** → Google popup → allow → **+ Add progress photo** → in your Drive: **FitPot Progress** folder with the photo.

Phase 5 fixes

Problem	Fix
🇺🇸 shows "USDA search limit reached"	Add your own key (Step 22), then re-run the deploy. Also check Step 24 (the workflow lines).
📦 results slow or "search failed"	Open Food Facts is a free community service and is sometimes slow. Try again; 🇮🇳 and ⭐ results still work.
Barcode not found	Common for Indian products. Tap **Create food**; next time it scans instantly from My foods.
Scanner says camera denied	Allow camera for the site (see Phase 4 fixes). You can also type the barcode number.
Progress photos say "aren't switched on yet"	`VITE_GOOGLE_CLIENT_ID` secret missing. Step 23, then re-run the deploy.
Google popup: "Error 400: redirect_uri_mismatch" / "origin not allowed"	Step 23.6: the origin must be exactly `https://kriticparikh.github.io` (no path, no trailing slash).
Google popup: "Access blocked: app has not completed verification" / only you can connect	Consent screen still in Testing. Step 23.5: **Publish app**.
Popup blocked	Allow pop-ups for the site; the Google window must open from a tap.
A dish's serving size looks off (e.g. a huge "plate")	Serving sizes come from the INDB dataset. Switch the unit to **g** and enter your own amount.

About the Indian food data

INDB is published with an open-access research paper and a public GitHub repo, but the repo doesn't state an explicit licence. It's credited in the app. If you ever take FitPot beyond a friend group, confirm the licence with the authors first. The data lives in one file (`public/data/indb.json`), so it's easy to swap.

Phase 5.1 — Each member's own timezone + pot currency

Two changes for groups spread across countries (e.g. friends in India, the US, the UK and Canada).

What changed

1. Your day follows your own phone's timezone

- When you join (or create) a group, your phone's timezone is saved on your member record.
- **Your** "today", check-in, undo, workout photo, rest-day vote and the photo watermark all use **your** local day: midnight to midnight where you are.
- **The group settles a day only once it has ended for everyone**, i.e. after the midnight of the member furthest behind (furthest west). Weekly groups settle once Sunday has ended for everyone.
    - Example: you're in New York, Asha is in India. At 10 PM Friday in New York it's already 7:30 AM Saturday for Asha. Asha's Friday is over (she either checked in or missed), but the group's Friday settles only at **your** midnight. On the Board it then shows "Settled through Fri".
- **Today tab:** the crew list shows each friend's status for your day (**In ✓** / **Not yet** / **Missed** / **Not started**) and their local time ("6:30 AM Sat there").
- **Wall:** photo times show in the poster's own local time, with their zone if it differs from yours (e.g. "7:05 AM IST").
- **Group tab:** each member's timezone and current local time.
- **Travelling or moving:** if your phone's timezone changes, the app notices when you open it and switches you over **from your next day**. The day you're in never stretches or shrinks. Changes are logged permanently. Today shows a line like "switching to IST from tomorrow".
- **Group settings** (penalty, target, currency) start from the day after the **furthest-ahead** member's today, so nobody's day changes once it has started.
- Members who joined before 5.1 have no saved timezone and use the group's timezone. If their phone is somewhere else, the app logs the change automatically and it applies from their next day.
- **Food log and progress** (Me tab) use your own day too.

2. Pot currency: ₹ INR · $ USD · £ GBP · C$ CAD

- Pick it when **creating** a group (pre-selected from your phone's timezone: India → ₹, UK → £, Canada → C$, else $). The default penalty follows: ₹100 or 5 in the others.
- **Changing it: Group → Pot currency → Switch to…**
    1. Choose the new currency. FitPot fetches **today's exchange rate** (European Central Bank rates via the free Frankfurter service; a second free source is used if that fails). You can edit the rate.

2. The new penalty is pre-filled at the same rate (₹100 × 0.0118 = $1.18). Round it to whatever the group agrees.
3. **Switch** → confirm. It starts **tomorrow** (same rule as penalty changes) and is logged permanently with the rate.

- **From the switch day, every amount shows in the new currency, including all past months**, converted at the saved rate. Three months of ₹ balances become $ at that day's rate. The Board shows a note like "From Oct 10: 1 ₹ = 0.0118 $".
- Switching again chains the rates (₹ → $ → £ multiplies both saved rates). Nothing stored is rewritten; the conversion is recalculated from the log, like every balance.
- Existing groups start as **$ USD**. If your group's numbers were always meant as rupees, switch to ₹ and set the rate to **1**: that relabels the old amounts as ₹ without converting them (Step 28).
- Penalty limit raised to 100,000 so rupee penalties fit.

Data added

```
groups/{groupId}                          + currency (base, set at creation)
groups/{groupId}/members/{uid}            + timezone (your phone's, at join;
locked)
groups/{groupId}/tzchanges/{id}           uid, timezone, effectiveDate (your next
day), setAt
groups/{groupId}/currencies/{id}          from, currency, rate (1 old = rate new),
penalty, effectiveDate, setBy, setAt
```

A currency switch also writes a normal `penalties` entry (the new penalty) in the same batch.

What you do

Step 26 — Update the Firestore rules. Firebase → **Firestore Database → Rules** → select all → delete → paste the new `firestore.rules` → **Publish.** New in this phase: the `tzchanges` and `currencies` logs, the member `timezone` (locked once set), group `currency`, and the higher penalty limit. **Do this before uploading the code**, or the app's automatic timezone log will be refused.

Step 27 — Upload the new code. See "Uploading an update". Drag in: `src`, `docs`, `firestore.rules` → **Commit changes.** No new secrets, no Apps Script change, no new libraries.

Step 28 — (Optional) set your existing group's currency. Groups created before 5.1 start as **$ USD**. In the app: **Group → Pot currency → Switch to…**

- Old amounts really were dollars → keep the fetched rate. Past amounts are converted at today's rate.
- Old amounts were always meant as rupees (the app just showed "$") → change the rate to **1**. Past amounts keep their numbers and just show as ₹. Set the penalty you want.
- **Switch**. It starts tomorrow.

Check it worked

1. **Group** tab → each member shows their timezone and local time, and there's a **Pot currency** card.
2. **Today** → the rule line ends with "your day runs on EDT" (or IST, etc.).
3. A friend in another country opens the app → their Today shows **their** date; on your Today they show their local time.
4. **Group → Pot currency → Switch to ₹** → a rate appears within a couple of seconds and the penalty is pre-filled → **Switch** → the card shows "→ ₹ from <tomorrow>".
5. The day after: **Board** amounts are in ₹ with the conversion note; **Today** shows the ₹ penalty.
6. Create a new test group from a phone set to India: the currency defaults to ₹ and the penalty to 100.

Phase 5.1 fixes

Problem	Fix
"Missing or insufficient permissions" when opening the app, switching currency, or after travelling	Rules not updated. Step 26.
"Couldn't fetch today's rate"	Both free rate services were unreachable. Type the rate in yourself (search "1 INR to USD").
Board says "Settled through" a day earlier than expected	Normal when someone in the group is further west: the day settles at their midnight.
A friend shows "Not started"	Your day hasn't begun for them yet (they're behind you).
My Today shows tomorrow's date / yesterday's	Your phone's timezone changed. It switches from your next day; the Today rule line says so.
Old group shows $ but we've always meant ₹	Step 28 with rate **1** (relabels without converting).
Photo folder names in Drive use New York dates	Folder names come from the Apps Script `TIMEZONE` property (organiser's). Only naming; the app uses each person's own day.

Phase 5.2 — Me first, several groups, Training (progressive overload)

What changed

1. Sign in → Me. Groups are optional.

- After sign-in you land on **Me** (now the first tab). Food, Training, Progress and Targets all work without a group.
- **Today, Calendar, Wall and Board** show **"Join or create a group"** until you're in one.
- **Group tab:** lists **your groups** (✓ Viewing / Switch), **+ Join or create another group**, then the settings of the group you're viewing.
- **Top bar:** a group switcher drop-down appears once you're in a group.

2. Several groups, one check-in.

- You can be in as many groups as you like. You're part of each pot **until you press Leave**, whichever group you're viewing.

- **One check-in counts in every group you're in.** One workout photo (or one tap, if no group requires a photo) checks you in everywhere. Each group gets its own copy of the photo, so groups never see each other's walls.
- If **any** of your groups requires a photo, CHECK IN opens the camera. Groups that require a photo are skipped (with a note) if photos aren't set up.
- **Undo** undoes today's check-in in all your groups.
- **Leave this group** (Group tab, bottom): you stop counting **from tomorrow**. Today still counts, and in a weekly group the week you've started still counts. Your past results stay on the Board, marked "(left)". Rejoining a group you left isn't supported yet.

3. Me → 🏋 Training (progressive overload). Five sections:

- 📝 **Log: + Start workout**, **↻ Repeat last**, or ▶ a saved routine. Add exercises from a built-in library (70+ exercises by muscle group, filtered to your equipment) or create your own. Each set is weight × reps.
  - Every exercise shows **last time** and a **"beat it"** target: one more rep, or a bit more weight once you hit 12 reps.
  - Tick ✓ on a set to start the **rest timer** (vibrates when done; +15 s / Skip).
  - **Notes** per workout. On save, 🏆 **new personal records** are celebrated.
  - The history list shows this week's training days vs your target, and PRs per workout.
- 💪 **Muscles:** week by week (‹ ›), for every muscle group: **sets** and **volume** (weight × reps) vs last week, and each exercise's **best set this week vs last week:** ▲ beat it, = matched, ▼ dropped, ★ new. This is the progressive-overload view.
- 🏆 **Records:** per exercise: heaviest weight, best **estimated 1-rep max**, most reps, and a chart of your estimated 1-rep max over time. (Estimated 1-rep max = weight × (1 + reps ÷ 30), the Epley formula, so 8 × 60 kg and 3 × 70 kg can be compared.) Filter by muscle group.
- 📋 **Routines:** save workouts you repeat (e.g. Push / Pull / Legs) with target sets × reps. Starting one pre-fills last time's weights.
- ⚙ **Setup:** main goal, days per week, session length, experience, rest-timer length, **equipment you have** (checklist + your own items, add/remove any time), and **limitations / injuries**. Your body-weight target is the one in Targets. All of this is stored for the Phase 8 plan model.

4. Me → Targets now also has:

- 📏 **Body measurements:** waist, chest, hips, arm, thigh, neck (cm or in) and body-fat %, by date, with change since your first entry.
- ⬇ **Export my data:** Workouts, Food, Weight, Measurements as **CSV** (opens in Excel / Google Sheets), or **Everything as JSON**.

Everything in Me is private (`private/{you}/…`, owner-only), as before.

Data added

```
users/{uid}                          + groupIds [all your groups]  (groupId =
the one you're viewing)
groups/{gid}/members/{uid}           + leftAt (server time, set once when you
Leave)
private/{uid}/settings/training      goal, daysPerWeek, sessionMin, experience,
restSec, limitations, equipment [keys], customEquipment [names]
private/{uid}/workouts/{id}          date, title, notes, exercises [{ key, name,
muscle, sets [{ reps, kg }] }]
private/{uid}/exercises/{id}         your own exercises: name, muscle, eq
private/{uid}/routines/{id}          name, items [{ key, name, muscle, sets,
reps }]
private/{uid}/measurements/{date}    date, cm { waist, chest, hips, arm, thigh,
neck }, bodyFat
private/{uid}/settings/main          + measureUnit (cm | in)
```

Weights are always stored in kg; lb users see lb.

What you do

Step 29 — Update the Firestore rules. Firebase → **Firestore Database → Rules** → select all → delete → paste the new `firestore.rules` → **Publish.** New: **Leave** (`leftAt`, set once, server time), and members who have left can no longer check in, post photos, vote rest days or change settings in that group.

Step 30 — Upload the new code. See "Uploading an update". Drag in: `src`, `docs`, `firestore.rules`, `README.md` → **Commit changes.** ⚠️ `src/screens/Onboarding.jsx` was moved to `src/components/JoinCreate.jsx`. Uploading doesn't delete files, so delete the old one on GitHub: open `src/screens/Onboarding.jsx` → ⋯ → **Delete file** → **Commit changes.** (Leaving it is harmless; nothing imports it.) No new secrets, no Apps Script change, no new libraries.

Check it worked

1. Sign in → you land on **Me**. The tab bar reads **Me · Today · Calendar · Wall · Board · Group**.
2. Your existing group shows in the top-bar switcher and under **Group → Your groups**.
3. On a fresh test account: **Today** says "Join or create a group".
4. Join a second group → the switcher lists both → Today says "One check-in counts in all 2 of your groups" → check in → both groups show you as **In** ✓.
5. **Me → Training → Setup** → pick your equipment → **Save setup**.

6. **Log → + Start workout → + Add exercise** → Barbell bench press → enter sets → tick ✓ (the rest timer runs) → **Save workout**.
7. Next time you add bench press, you see **Last time** and **beat it**. Beat it → 🏆 screen.
8. **Muscles** shows this week vs last week; **Records** shows the 1-rep max chart after two sessions.
9. **Targets → Body measurements** → save → shows latest. **Export my data → Workouts (CSV)** downloads a file.

Phase 5.2 fixes

Problem	Fix
"Missing or insufficient permissions" when leaving a group	Rules not updated. Step 29.
After checking in, one group says "needs a photo"	That group requires a photo and you checked in without one (no group you're in had the camera on, or photos aren't set up). Post a photo from that group's Wall.
I left a group but still see it in the Board history	By design: past results stay. It's removed from your switcher.
"Rejoining a group you left isn't supported yet"	Ask the group to create a new group, or wait for a later phase.
Exercise list is short	"Only what my equipment allows" is on. Untick it, or add equipment in Training → Setup.
Export does nothing on iPhone	iOS opens the file in a preview: tap **Share → Save to Files**.
Rest timer doesn't vibrate on iPhone	iPhones don't allow web apps to vibrate. Watch the bar turn orange.

Phase 5.3 — Email sign-in, verification, Account

What changed

1. Two ways to sign in. The login screen has **Continue with Google**, and below it **email + password**:

- **Create account:** your name (what friends see), email, password (at least 8 characters, 👁 to show/hide).

- **Sign in**, and **Forgot password?**: Firebase emails a reset link that brings you back to FitPot.
- Clear messages: wrong password, email already registered, too many attempts, no connection…
- **One account per email.** If you signed up with Google, a password sign-in with the same Gmail says to use Google. You can add a password later (see 3).

2. Email verification. New email accounts get a "confirm your email" link. Until it's clicked:

- **Me works fully**, with a 📧 banner (**I've confirmed** · **Resend email**).
- **Joining or creating a group is blocked.** The Group tab shows the banner instead of the join/create form, and the **Firestore rules** refuse it too. Google accounts count as verified.

3. ⚙️ **Account** (top-right gear; **Sign out** moved here):

- See your name, email, how you sign in, and whether your email is verified.
- **Change display name:** updates your profile and your name in every group you're in.
- **Change password** (email accounts): current + new password.
- **Add a password** (Google accounts): then you can sign in with Google **or** your Gmail + that password.
- **Delete my account:** confirm with your password (or Google), type `DELETE`. It:
  - deletes your login and everything in **Me** (food, training, weight, measurements, targets);
  - leaves every group you're in (and anonymises you in groups you left from 5.3 on); your **past check-ins stay**, so friends' balances don't change, and your name shows as **"Former member"**;
  - leaves your progress photos in your own Drive (your files) and workout photos in the organiser's Drive. Export your data first (Me → Targets) if you want a copy.

What you do

Step 31 — Turn on email sign-in in Firebase. Firebase console → **Authentication → Sign-in method → Add new provider → Email/Password** → switch on **Email/Password** (leave "Email link (passwordless sign-in)" off) → **Save.**

Step 32 — (Recommended) Make the emails look like FitPot. Authentication → Templates:

- **Email address verification** → ✏️ → **Sender name:** `FitPot` → **Save.**
- **Password reset** → ✏️ → **Sender name:** `FitPot` → **Save.** The emails come from `noreply@fitpot-grind.firebaseapp.com`; that's normal, and free. They can land in spam the first time.

Step 33 — Update the Firestore rules. Firestore Database → Rules → select all → delete → paste the new `firestore.rules` → **Publish.** (New: creating or joining a group needs a verified email.)

Step 34 — Upload the new code. Drag in `src`, `docs`, `firestore.rules`, `README.md` → **Commit changes.** No new secrets, no Apps Script change, no new libraries.

Check it worked

1. Sign out (⚙️ → Sign out) → the login screen shows Google **and** email.
2. **Create account** with a second email you own → you land on **Me** with the 📧 banner.
3. **Group** tab → shows "Confirm your email" instead of join/create.
4. Open the email → tap the link → back in FitPot → **I've confirmed** → the banner goes; Group now offers join/create.
5. Sign out → **Sign in** with the email and password → works. Try a wrong password → "Email or password is wrong…".
6. **Forgot password?** → reset email arrives → choose a new password → sign in with it.
7. On your Google account: ⚙️ → **Add a password** → sign out → sign in with your Gmail + that password → same account, same groups.
8. On a throwaway test account: ⚙️ → **Delete my account…** → the account is gone; in its group it shows as "Former member".

Phase 5.3 fixes

Problem	Fix
"Email sign-in isn't switched on yet"	Step 31.
No verification / reset email	Check spam. Then ⚙ → **Resend email**. Firebase limits how often it sends; wait a minute between tries.
"I've confirmed" says not confirmed yet	Open the link from the **latest** email (older links expire), then press it again.
"Missing or insufficient permissions" when joining or creating a group	Email not verified yet, or the rules weren't updated (Step 33).
"Email or password is wrong" but you're sure	That email may use Google sign-in. Use **Continue with Google**, then ⚙ → **Add a password**.
"That email already has an account" when creating one	Sign in instead, or use Google if that's how you signed up.
"For your security, sign in again and retry"	Firebase wants a recent sign-in for password changes and deletion. Sign out and in, then retry.
A deleted account's name still shows in a group it left before Phase 5.3	Older leaves weren't recorded on the profile. It only affects the name shown on old history.
Reset/verify link opens a Firebase page instead of FitPot	The link works either way; the "Continue" button returns to FitPot. Check `kriticparikh.github.io` is in Authorized domains (Step 3.3).

Uploading an update (website method)

Each update comes as a zip. To apply it:

1. Unzip it on your computer.
2. On GitHub, open the folder where `package.json` lives (the repo root, or `fitpot/` if your files are in that folder).
3. **Add file → Upload files** → drag in the files and folders from the zip (usually the `src` folder, `docs` folder, `firestore.rules` and `README.md`; each update's section says

which). Uploading a folder with the same name replaces the files inside it.

4. **Commit changes**. The Actions tab starts a deploy automatically; wait for the green check (about 1 minute).
5. Reload the app. An installed app updates itself on the next open or two.

Appendix A — deploy.yml

If your files are at the repo root, use this as-is. If they're inside a `fitpot/` folder, uncomment the two lines marked 👉.

yaml

```yaml
name: Deploy to GitHub Pages

on:
  push:
    branches: [main]
  workflow_dispatch:

permissions:
  contents: read
  pages: write
  id-token: write

concurrency:
  group: pages
  cancel-in-progress: true

jobs:
  build:
    runs-on: ubuntu-latest
    # 👉 defaults: { run: { working-directory: fitpot } }
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 22
      - run: npm install
      - run: npm run build
        env:
          BASE_PATH: /${{ github.event.repository.name }}/
          VITE_FIREBASE_API_KEY: ${{ secrets.VITE_FIREBASE_API_KEY }}
          VITE_FIREBASE_AUTH_DOMAIN: ${{ secrets.VITE_FIREBASE_AUTH_DOMAIN }}
          VITE_FIREBASE_PROJECT_ID: ${{ secrets.VITE_FIREBASE_PROJECT_ID }}
          VITE_FIREBASE_STORAGE_BUCKET: ${{ secrets.VITE_FIREBASE_STORAGE_BUCKET }
          VITE_FIREBASE_MESSAGING_SENDER_ID: ${{ secrets.VITE_FIREBASE_MESSAGING_S
          VITE_FIREBASE_APP_ID: ${{ secrets.VITE_FIREBASE_APP_ID }}
          VITE_DRIVE_UPLOAD_URL: ${{ secrets.VITE_DRIVE_UPLOAD_URL }}
          VITE_USDA_API_KEY: ${{ secrets.VITE_USDA_API_KEY }}
          VITE_GOOGLE_CLIENT_ID: ${{ secrets.VITE_GOOGLE_CLIENT_ID }}
      - uses: actions/upload-pages-artifact@v3
        with:
          path: dist          # 👉 fitpot/dist
```

```
  deploy:
    needs: build
    runs-on: ubuntu-latest
    environment:
      name: github-pages
      url: ${{ steps.deployment.outputs.page_url }}
    steps:
      - id: deployment
        uses: actions/deploy-pages@v4
```

Appendix B — Run locally (optional)

bash

```
cp .env.example .env.local   # paste the Firebase values
npm install
npm run dev                  # opens http://localhost:5173
```

Add `localhost` to Firebase Authorized domains if it isn't there already (it usually is by default).